BLAKE3 검증 시스템 분석

개요

iroh-blob의 BLAKE3 검증 시스템은 데이터 무결성을 보장하기 위한 핵심 구성 요소입니다. 이 시스템은 BAO (BLAKE3 Verified Streaming) 프로토콜을 기반으로 하여 스트리밍 중에도 실시간으로 데이터 검증이 가능하도록 설계되었습니다.

BAO (BLAKE3 Verified Streaming) 구현 원리

BAO의 기본 개념

BAO는 BLAKE3 해시 함수의 특별한 속성을 활용하여 대용량 데이터를 스트리밍하면서 동시에 검증할 수 있게 해주는 프로토콜입니다.

```
// Hash 구조체 - BLAKE3 해시의 래퍼
pub struct Hash(blake3::Hash);


impl Hash {
/// 빈 바이트 범위의 해시 상수
pub const EMPTY: Hash = Hash::from_bytes([
175, 19, 73, 185, 245, 249, 161, 166, 160, 64, 77, 234, 54, 220, 201, 73,
155, 203, 37, 201, 173, 193, 18, 183, 204, 154, 147, 202, 228, 31, 50, 98,
]);
```

```
/// 제공된 바이트의 해시 계산
pub fn new(buf: impl AsRef<[u8]>) -> Self {
let val = blake3::hash(buf.as_ref());
Hash(val)
}
}
```

BAO 트리 구조

BAO는 데이터를 청크 단위로 분할하고 이를 트리 구조로 조직화합니다:

1. **청크 크기**: 1024바이트 (1KiB) 단위
2. **청크 그룹**: 16개 청크 = 16KiB 단위
3. **트리 구조**: 이진 트리 형태의 해시 트리

```
/// iroh에서 사용하는 블록 크기: 2^4*1024 = 16KiB
pub const IROH_BLOCK_SIZE: BlockSize = BlockSize::from_chunk_log(4);
```

청크 단위 해시 트리 구조와 검증 과정

트리 구조 생성

데이터는 다음과 같은 계층 구조로 해시됩니다:

1. **리프 노드**: 1024바이트 청크의 BLAKE3 해시
2. **내부 노드**: 두 자식 노드 해시의 조합
3. **루트 노드**: 전체 데이터의 최종 해시

해시 계산 과정

```
fn hash_subtree(start_chunk: u64, data: &[u8], is_root: bool) -> blake3::Hash {
use blake3::hazmat::{ChainingValue, HasherExt};
if is_root {
debug_assert!(start_chunk == 0);
blake3::hash(data)
} else {
let mut hasher = blake3::Hasher::new();
hasher.set_input_offset(start_chunk * 1024);
hasher.update(data);
let non_root_hash: ChainingValue = hasher.finalize_non_root();
blake3::Hash::from(non_root_hash)
}
}
```

부모 노드 해시 계산

```
fn parent_cv(
left_child: &blake3::Hash,
right_child: &blake3::Hash,
is_root: bool,
) -> blake3::Hash {
```

```
use blake3::hazmat::{merge_subtrees_non_root, merge_subtrees_root, ChainingValue

let  left_child: ChainingValue = *left_child.as_bytes();

let  right_child: ChainingValue = *right_child.as_bytes();

if  is_root {

merge_subtrees_root(&left_child, &right_child, Mode::Hash)

} else {

blake3::Hash::from(merge_subtrees_non_root(

&left_child,

&right_child,

Mode::Hash,

))

}

}
```

부분 데이터 검증과 스트리밍 검증의 차이점

부분 데이터 검증

부분 데이터 검증은 전체 파일의 일부분만 다운로드한 상태에서도 해당 부분의 무결성을 확인할 수 있는 기능입니다.

특징:

- **청크 단위 검증**: 각 1024바이트 청크를 독립적으로 검증
- **범위 요청 지원**: 특정 바이트 범위만 요청하고 검증 가능

- **트리 경로 검증**: 요청한 청크에서 루트까지의 해시 경로만 필요

```
/// 청크 범위를 효율적으로 표현하는 타입
pub  type ChunkRanges = RangeSet2<ChunkNum>;


impl ChunkRangesExt for ChunkRanges {
/// 바이트 범위를 청크 범위로 변환
fn  bytes(ranges: impl RangeBounds<u64>) -> Self {
round_up_to_chunks(&bounds_from_range(ranges, |v| v))
}
/// 단일 청크 범위 생성
fn  chunk(offset: u64) -> Self {
ChunkRanges::from(ChunkNum(offset)..ChunkNum(offset + 1))
}
}
```

스트리밍 검증

스트리밍 검증은 데이터를 수신하는 동시에 실시간으로 검증하는 방식입니다.

특징:

- **실시간 검증**: 데이터 수신과 동시에 해시 검증 수행
- **최대 지연**: 16KiB 청크 그룹 단위로 최대 지연 발생
- **메모리 효율성**: 전체 데이터를 메모리에 보관하지 않고 검증 가능

```rust
/// BAO 콘텐츠 아이템 - 스트리밍 중 전송되는 데이터 단위
pub enum BaoContentItem {
/// 부모 노드 (해시 쌍)
Parent(Parent),
/// 리프 노드 (실제 데이터)
Leaf(Leaf),
}

/// 리프 노드 구조
pub struct Leaf {
pub offset: u64, // 데이터 오프셋
pub data: Bytes, // 실제 데이터
}
```

16KiB 청크 그룹 최적화 기법

청크 그룹의 목적

16KiB 청크 그룹은 다음과 같은 최적화를 제공합니다:

1. **메타데이터 오버헤드 감소**: 16개 청크를 하나의 단위로 처리
2. **I/O 효율성**: 더 큰 단위의 읽기/쓰기 작업
3. **네트워크 효율성**: 더 적은 수의 메시지로 데이터 전송

청크 그룹 처리 과정

```rust
/// 청크 그룹 단위로 데이터 처리
async fn init_impl<W, P>(
tree: BaoTree,
mut data: impl Read,
outboard: &mut PreOrderOutboard<W>,
buffer: &mut [u8],
progress: &mut P,
) -> io::Result<std::result::Result<(), P::Error>>
where
W: WriteAt,
P: Sink<ChunkNum>,
{
let mut stack = SmallVec::<[blake3::Hash; 10]>::new();
for item in tree.post_order_chunks_iter() {
match item {
BaoChunk::Parent { is_root, node, .. } => {
let right_hash = stack.pop().unwrap();
let left_hash = stack.pop().unwrap();
outboard.save(node, &(left_hash, right_hash))?;
let parent = parent_cv(&left_hash, &right_hash, is_root);
stack.push(parent);
}
```

```
BaoChunk::Leaf {
size,
is_root,
start_chunk,
..
} => {
if let Err(err) = progress.send(start_chunk).await {
return Ok(Err(err));
}
let buf = &mut buffer[..size];
data.read_exact(buf)?;
let hash = hash_subtree(start_chunk.0, buf, is_root);
stack.push(hash);
}
}
}
Ok(Ok(()))
}
```

최적화 효과

1. **메모리 사용량 최적화**:

- 청크 그룹 단위로 버퍼링하여 메모리 사용량 제한
- 스트리밍 처리로 대용량 파일도 효율적 처리

2. **I/O 최적화**:

- 16KiB 단위의 읽기/쓰기로 시스템 콜 횟수 감소
- 버퍼링을 통한 디스크 I/O 효율성 향상

3. **네트워크 최적화**:

- 더 큰 메시지 단위로 네트워크 오버헤드 감소
- QUIC 스트림의 효율적 활용

검증 과정의 실제 구현

데이터 수신 및 검증

```
/// BAO 데이터 가져오기 및 검증
async fn import_bao(
entry: BaoFileHandle,
size: NonZeroU64,
mut stream: mpsc::Receiver<BaoContentItem>,
tx: irpc::channel::oneshot::Sender<api::Result<()>>,
) {
let size = size.get();
let tree = BaoTree::new(size, IROH_BLOCK_SIZE);
while let Some(item) = stream.recv().await.unwrap() {
entry.0.state.send_if_modified(|state| {
match item {
BaoContentItem::Parent(parent) => {
```

```
// 부모 노드 해시 저장
if let Some(offset) = tree.pre_order_offset(parent.node) {
let mut pair = [0u8; 64];
pair[..32].copy_from_slice(parent.pair.0.as_bytes());
pair[32..].copy_from_slice(parent.pair.1.as_bytes());
partial.outboard.write_at(offset * 64, &pair)
.expect("writing to mem can never fail");
}
false
}
BaoContentItem::Leaf(leaf) => {
// 리프 데이터 저장 및 검증
let start = leaf.offset;
partial.data.write_at(start, &leaf.data)
.expect("writing to mem can never fail");
let added = chunk_range(&leaf);
let update = partial.bitfield.update(&Bitfield::new(added.clone(), size));
if update.new_state().complete {
// 완전한 데이터 수신 시 상태 변경
let data = std::mem::take(&mut partial.data);
let outboard = std::mem::take(&mut partial.outboard);
*state = CompleteStorage::new(data.into(), outboard.into()).into();
}
```

```
update.changed()
}
}
});
}
}
```

검증 실패 처리

검증 시스템은 다양한 오류 상황을 처리합니다:

```
/// 디코딩 오류 타입
pub  enum DecodeError {
/// 청크를 찾을 수 없거나 유효하지 않음
ChunkNotFound {},
/// 부모 노드를 찾을 수 없음
ParentNotFound { node: TreeNode },
/// 리프 노드를 찾을 수 없음
LeafNotFound { num: ChunkNum },
/// 부모 해시 불일치
ParentHashMismatch { node: TreeNode },
/// 리프 해시 불일치
LeafHashMismatch { num: ChunkNum },
/// 스트림 읽기 오류
```

```
Read { source: endpoint::ReadError },

/// 일반적인 I/O 오류

DecodeIo { source: io::Error },

}
```

성능 특성 및 보안 고려사항

성능 특성

1. **검증 속도**: BLAKE3의 높은 성능으로 검증이 병목이 되지 않음
2. **메모리 효율성**: 스트리밍 검증으로 메모리 사용량 최소화
3. **병렬 처리**: 청크 단위 독립 검증으로 병렬화 가능

보안 고려사항

1. **강력한 무결성 보장**: BLAKE3의 암호학적 보안성
2. **실시간 검증**: 데이터 수신과 동시에 무결성 확인
3. **부분 검증**: 일부 데이터만으로도 해당 부분의 무결성 보장
4. **공격 방어**: 해시 불일치 시 즉시 연결 종료

결론

iroh-blob의 BLAKE3 검증 시스템은 BAO 프로토콜을 통해 효율적이고 안전한 데이터 검증을 제공합니다. 16KiB 청크 그룹 최적화와 스트리밍 검증을 통해 대용량 데이터도 메모리 효율적으로 처리하면서 강력한 무결성 보장을 제공합니다. 이는 분산 시스템에서 데이터 무결성이 중요한 환경에 매우 적합한 설계입니다.